CSE 4851
Design Pattern

Assignment 02

Implementing Abstract Factory, Builder, Composite, Singleton, Facade, Strategy and Prototype Pattern for the Age of Villagers (AoV) Village Creation Module

---

Name	:	Hasib Altaf
Student ID	:	210041102
Section	:	1B
Submission Date	:	August 9, 2026

1 Scenario

Age of Villagers (AoV) is a village-building game. The scope of this assignment is the **village creation module**: a player selects a theme, and the system must automatically create the corresponding house, tree, and water source for that theme.

The module must satisfy:

- A **Modern Village** theme that creates a Brick House, a Mango Tree, and a Swimming Pool.
- A **Traditional Village** theme that creates a Mud House, a Banana Tree, and a Pond.
- Every house, tree, and water source must itself be assembled from **simple shapes** (rectangles, circles, triangles, squares, semicircles), not hard-coded as one monolithic object.
- **New themes** (e.g. Desert Village, Forest Village) must be addable **without modifying any existing code** – i.e. the design must respect the Open/Closed Principle.

To satisfy these requirements, seven design patterns are used together:

- **Abstract Factory Pattern** for creating a theme's whole family of objects (house + tree + water source) consistently.
- **Builder Pattern** for assembling each object step-by-step out of simple shapes.
- **Composite Pattern** for treating a single shape and a whole house/tree/water-source uniformly.
- **Singleton Pattern** for a single, global registry that maps a theme name to its factory.
- **Facade Pattern** for a single, simple entry point that hides the registry/factory/builder machinery from client code.
- **Strategy Pattern** for decoupling how a village object is rendered/displayed from the object itself.
- **Prototype Pattern** for cheaply cloning an already-built village object instead of rebuilding it from scratch.

2 Pattern Usage

2.1 Abstract Factory Pattern

The Abstract Factory Pattern is the core pattern of this assignment. It declares creation methods for a whole *family* of related products (`House`, `Tree`, `WaterSource`) that must all belong to the same theme, so a factory can never accidentally mix, e.g., a Brick House with a Banana Tree.

- `VillageThemeFactory` – the abstract factory interface.
- `ModernVillageFactory` – produces Brick House + Mango Tree + Swimming Pool.
- `TraditionalVillageFactory` – produces Mud House + Banana Tree + Pond.
- `DesertVillageFactory` – produces Adobe House + Cactus Tree + Oasis, added purely to *prove* that new themes need no changes to existing classes.

2.2 Builder Pattern

Since every house, tree, and water source must be built by adding different simple shapes, the Builder Pattern separates the step-by-step assembly process (foundation → structure → details) from the final object's representation. A `VillageObjectDirector` owns the fixed construction order so it never needs to be repeated in each concrete builder.

- `VillageObjectBuilder` – the builder interface (`buildFoundation`, `buildStructure`, `buildDetails`).
- `VillageObjectDirector` – runs the builder steps in the correct order.

- Nine concrete builders (three per theme), e.g. `BrickHouseBuilder`, `MangoTreeBuilder`, `SwimmingPoolBuil` `MudHouseBuilder`, `BananaTreeBuilder`, `PondBuilder`, `AdobeHouseBuilder`, `CactusTreeBuilder`, `OasisBuilder`.

2.3 Composite Pattern

A house, tree, or water source is never a single shape – it is a collection of shapes (walls, roof, door, foliage, fruit, water surface, tiles, etc.). The Composite Pattern lets `CompositeShape` implement the same `Shape` interface (`getArea()`, `draw()`) as its individual children, so a single brick and an entire brick house can be handled uniformly, and area/drawing calls recurse automatically over nested parts.

- `Shape` – the common component interface.
- `Rectangle`, `Square`, `Triangle`, `Circle`, `SemiCircle` – simple/leaf shapes.
- `CompositeShape` – the composite; `VillageObject` extends it.

2.4 Singleton Pattern

The game needs exactly one authoritative place that knows, given a theme name, which factory should handle it. `VillageFactoryRegistry` is a thread-safe, lazily-initialized Singleton that themes register themselves into at startup and that is queried whenever a village is created.

- `VillageFactoryRegistry` – the Singleton registry.

2.5 Facade Pattern

`VillageCreationService` wraps registry lookup, abstract-factory calls, and builder/director machinery behind a single method, `createVillage(name, theme)`, so the rest of the game never needs to know about builders, directors, or the registry at all.

- `VillageCreationService` – the facade.
- `Village` – the simple data holder returned by the facade.

2.6 Strategy Pattern

How a village object is displayed is deliberately decoupled from the object itself. Today only `ConsoleRenderStrategy` exists (it prints an indented tree of shapes plus total area), but a `GraphicalRenderStrat` or `JsonExportRenderStrategy` could be introduced later without touching `VillageObject`, `House`, `Tree`, or `WaterSource`.

- `RenderStrategy` – the strategy interface.
- `ConsoleRenderStrategy` – the concrete console strategy.

2.7 Prototype Pattern

Each concrete village object (`BrickHouse`, `MangoTree`, etc.) has a private copy constructor and implements `deepClone()`, letting the game cheaply duplicate an already-built object (e.g. "plant another mango tree") instead of re-running the whole Builder → Director → Factory pipeline.

- `VillageObject.deepClone()` – the abstract prototype hook.
- `CompositeShape.cloneChildren()` – deep-copies every nested shape.

3 Pattern Collaboration

Workflow:

1. At startup, `Main` registers each available `VillageThemeFactory` (Modern, Traditional, Desert, ...) into the `VillageFactoryRegistry` Singleton.

Pattern	Responsibility
Abstract Factory	Creating a consistent family of themed objects
Builder	Step-by-step shape assembly of one object
Composite	Treating simple shapes and whole objects uniformly
Singleton	Single global lookup of theme → factory
Facade	One simple entry point for village creation
Strategy	Decoupled rendering/display of village objects
Prototype	Cheap duplication of an existing village object

2. Client code asks `VillageCreationService` (Facade) to `createVillage(name, theme)`.
3. The facade asks the registry for the right `VillageThemeFactory` (Abstract Factory) for that theme.
4. The factory delegates shape assembly to a `VillageObjectDirector` and the matching `VillageObjectBuild` (Builder), which adds `Rectangle`/`Circle`/`Triangle`/... shapes to build up a `House`, `Tree`, or `WaterSource` (Composite).
5. The resulting village objects can be duplicated at any time via `deepClone()` (Prototype).
6. Village objects are displayed through a `RenderStrategy` (Strategy), e.g. `ConsoleRenderStrategy`.

Extending with a new theme

Adding a brand-new theme (e.g. Forest Village) requires only:

1. New concrete products (e.g. `TreehouseHome`, `OakTree`, `Waterfall`).
2. New matching builders.
3. One new `VillageThemeFactory` implementation (e.g. `ForestVillageFactory`).
4. One extra registration line at the composition root: `registry.registerFactory(new ForestVillageFacto`

No existing file needs to change – this is demonstrated concretely in this project by the `Desert` theme, which was added exactly this way.

4 Implementation

The complete source tree (40 Java files across 8 packages) is listed below, grouped by package, in the same order in which the classes collaborate: shapes → village-object products → builders → abstract factories → registry → render strategy → facade → demo/composition root.

4.1 shape package – Composite Pattern (Simple Shapes)

`Shape.java`

```java
package com.aov.village.shape;

/**
 * The uniform interface for every drawable element in the game -
 * both a single primitive shape (Rectangle, Circle, ...) and a
 * composite object made of many shapes (House, Tree, ...).
 * This is the "Component" role of the Composite Pattern.
 */
public interface Shape {
    String getName();
    double getArea();
    void draw(int indentLevel);

    /** Prototype Pattern hook: every shape must know how to copy itself.
       */
```

```
    Shape cloneShape();
}
```

ShapeUtils.java

```
package com.aov.village.shape;

final class ShapeUtils {
    private ShapeUtils() { }

    static String indent(int level) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < level; i++) sb.append("  ");
        return sb.toString();
    }
}
```

CompositeShape.java

```
package com.aov.village.shape;

import java.util.ArrayList;
import java.util.List;

/**
 * COMPOSITE PATTERN
 * ------------------
 * Represents any village object (House, Tree, WaterSource, ...) as a
 * collection of simple shapes, while still exposing the same Shape
 * interface (getArea/draw) as its individual parts. This lets client
 * code treat "one brick" and "a whole brick house" uniformly.
 */
public abstract class CompositeShape implements Shape {
    protected final String name;
    protected final List<Shape> children = new ArrayList<>();

    protected CompositeShape(String name) {
        this.name = name;
    }

    public void addShape(Shape shape) {
        children.add(shape);
    }

    public void removeShape(Shape shape) {
        children.remove(shape);
    }

    public List<Shape> getChildren() {
        return children;
    }

    @Override public String getName() { return name; }

    @Override public double getArea() {
        double total = 0;
        for (Shape s : children) total += s.getArea();
        return total;
    }
```

```
41
42      @Override public void draw(int indentLevel) {
43          System.out.printf("%s+ %s (total area=%.2f)%n", ShapeUtils.indent(
                indentLevel), name, getArea());
44          for (Shape s : children) s.draw(indentLevel + 1);
45      }
46
47      /** PROTOTYPE PATTERN helper: deep-copies every child shape for cloning
            . */
48      protected List<Shape> cloneChildren() {
49          List<Shape> copy = new ArrayList<>();
50          for (Shape s : children) copy.add(s.cloneShape());
51          return copy;
52      }
53  }
```

Rectangle.java

```
1   package com.aov.village.shape;
2
3   public class Rectangle implements Shape {
4       private final double width;
5       private final double height;
6       private final String color;
7
8       public Rectangle(double width, double height, String color) {
9           this.width = width;
10          this.height = height;
11          this.color = color;
12      }
13
14      @Override public String getName() { return "Rectangle"; }
15
16      @Override public double getArea() { return width * height; }
17
18      @Override public void draw(int indentLevel) {
19          System.out.printf("%s- %s Rectangle [%sx%s], area=%.2f%n",
20                  ShapeUtils.indent(indentLevel), color, width, height,
                        getArea());
21      }
22
23      @Override public Shape cloneShape() { return new Rectangle(width,
            height, color); }
24  }
```

Square.java

```
1   package com.aov.village.shape;
2
3   public class Square implements Shape {
4       private final double side;
5       private final String color;
6
7       public Square(double side, String color) {
8           this.side = side;
9           this.color = color;
10      }
11
12      @Override public String getName() { return "Square"; }
```

```
13
14      @Override public double getArea() { return side * side; }
15
16      @Override public void draw(int indentLevel) {
17          System.out.printf("%s- %s Square [side=%s], area=%.2f%n",
18                  ShapeUtils.indent(indentLevel), color, side, getArea());
19      }
20
21      @Override public Shape cloneShape() { return new Square(side, color); }
22  }
```

Triangle.java

```
1   package com.aov.village.shape;
2
3   public class Triangle implements Shape {
4       private final double base;
5       private final double height;
6       private final String color;
7
8       public Triangle(double base, double height, String color) {
9           this.base = base;
10          this.height = height;
11          this.color = color;
12      }
13
14      @Override public String getName() { return "Triangle"; }
15
16      @Override public double getArea() { return 0.5 * base * height; }
17
18      @Override public void draw(int indentLevel) {
19          System.out.printf("%s- %s Triangle [base=%s, height=%s], area=%.2f%
            n",
20                  ShapeUtils.indent(indentLevel), color, base, height,
                        getArea());
21      }
22
23      @Override public Shape cloneShape() { return new Triangle(base, height,
            color); }
24  }
```

Circle.java

```
1   package com.aov.village.shape;
2
3   public class Circle implements Shape {
4       private final double radius;
5       private final String color;
6
7       public Circle(double radius, String color) {
8           this.radius = radius;
9           this.color = color;
10      }
11
12      @Override public String getName() { return "Circle"; }
13
14      @Override public double getArea() { return Math.PI * radius * radius; }
15
16      @Override public void draw(int indentLevel) {
```

```
        System.out.printf("%s- %s Circle [radius=%s], area=%.2f%n",
                ShapeUtils.indent(indentLevel), color, radius, getArea());
    }

    @Override public Shape cloneShape() { return new Circle(radius, color); }
}
```

SemiCircle.java

```
package com.aov.village.shape;

public class SemiCircle implements Shape {
    private final double radius;
    private final String color;

    public SemiCircle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    @Override public String getName() { return "SemiCircle"; }

    @Override public double getArea() { return 0.5 * Math.PI * radius * radius; }

    @Override public void draw(int indentLevel) {
        System.out.printf("%s- %s SemiCircle [radius=%s], area=%.2f%n",
                ShapeUtils.indent(indentLevel), color, radius, getArea());
    }

    @Override public Shape cloneShape() { return new SemiCircle(radius, color); }
}
```

4.2 object package – Composite root and abstract products

VillageObject.java

```
package com.aov.village.object;

import com.aov.village.shape.CompositeShape;
import com.aov.village.shape.Shape;

/**
 * The abstract "product" root that every concrete village object
 * (House, Tree, WaterSource) descends from. Serves as:
 *  - the Composite root (built out of Shapes),
 *  - the common return type used by the Abstract Factory,
 *  - the Prototype participant (deepClone()).
 */
public abstract class VillageObject extends CompositeShape {

    protected final String theme;

    protected VillageObject(String name, String theme) {
        super(name);
        this.theme = theme;
    }
```

```
21
22      public String getTheme() { return theme; }
23
24      /** e.g. "House", "Tree", "Water Source" */
25      public abstract String getType();
26
27      /** PROTOTYPE PATTERN: clone this object (and every shape inside it)
            cheaply. */
28      public abstract VillageObject deepClone();
29
30      /** Bridges the generic Shape#cloneShape() contract onto the more
            specific deepClone(). */
31      @Override public Shape cloneShape() { return deepClone(); }
32  }
```

House.java

```
1   package com.aov.village.object;
2
3   public abstract class House extends VillageObject {
4       protected House(String name, String theme) { super(name, theme); }
5       @Override public String getType() { return "House"; }
6   }
```

Tree.java

```
1   package com.aov.village.object;
2
3   public abstract class Tree extends VillageObject {
4       protected Tree(String name, String theme) { super(name, theme); }
5       @Override public String getType() { return "Tree"; }
6   }
```

WaterSource.java

```
1   package com.aov.village.object;
2
3   public abstract class WaterSource extends VillageObject {
4       protected WaterSource(String name, String theme) { super(name, theme);
            }
5       @Override public String getType() { return "Water Source"; }
6   }
```

4.3 object.modern package – Modern theme products

modern/BrickHouse.java

```
1   package com.aov.village.object.modern;
2
3   import com.aov.village.object.House;
4   import com.aov.village.object.VillageObject;
5
6   public class BrickHouse extends House {
7       public BrickHouse() { super("Brick House", "Modern"); }
8
9       private BrickHouse(BrickHouse other) {
10          super("Brick House", "Modern");
```

```
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new BrickHouse(this
        ); }
}
```

modern/MangoTree.java

```
package com.aov.village.object.modern;

import com.aov.village.object.Tree;
import com.aov.village.object.VillageObject;

public class MangoTree extends Tree {
    public MangoTree() { super("Mango Tree", "Modern"); }

    private MangoTree(MangoTree other) {
        super("Mango Tree", "Modern");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new MangoTree(this)
        ; }
}
```

modern/SwimmingPool.java

```
package com.aov.village.object.modern;

import com.aov.village.object.VillageObject;
import com.aov.village.object.WaterSource;

public class SwimmingPool extends WaterSource {
    public SwimmingPool() { super("Swimming Pool", "Modern"); }

    private SwimmingPool(SwimmingPool other) {
        super("Swimming Pool", "Modern");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new SwimmingPool(
        this); }
}
```

4.4 object.traditional package – Traditional theme products

traditional/MudHouse.java

```
package com.aov.village.object.traditional;

import com.aov.village.object.House;
import com.aov.village.object.VillageObject;

public class MudHouse extends House {
    public MudHouse() { super("Mud House", "Traditional"); }

    private MudHouse(MudHouse other) {
```

```
        super("Mud House", "Traditional");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new MudHouse(this);
        }
}
```

traditional/BananaTree.java

```
package com.aov.village.object.traditional;

import com.aov.village.object.Tree;
import com.aov.village.object.VillageObject;

public class BananaTree extends Tree {
    public BananaTree() { super("Banana Tree", "Traditional"); }

    private BananaTree(BananaTree other) {
        super("Banana Tree", "Traditional");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new BananaTree(this
        ); }
}
```

traditional/Pond.java

```
package com.aov.village.object.traditional;

import com.aov.village.object.VillageObject;
import com.aov.village.object.WaterSource;

public class Pond extends WaterSource {
    public Pond() { super("Pond", "Traditional"); }

    private Pond(Pond other) {
        super("Pond", "Traditional");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new Pond(this); }
}
```

4.5 object.desert package – Desert theme products (extensibility proof)

desert/AdobeHouse.java

```
package com.aov.village.object.desert;

import com.aov.village.object.House;
import com.aov.village.object.VillageObject;

public class AdobeHouse extends House {
    public AdobeHouse() { super("Adobe House", "Desert"); }

    private AdobeHouse(AdobeHouse other) {
```

```
        super("Adobe House", "Desert");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new AdobeHouse(this
        ); }
}
```

desert/CactusTree.java

```
package com.aov.village.object.desert;

import com.aov.village.object.Tree;
import com.aov.village.object.VillageObject;

public class CactusTree extends Tree {
    public CactusTree() { super("Cactus Tree", "Desert"); }

    private CactusTree(CactusTree other) {
        super("Cactus Tree", "Desert");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new CactusTree(this
        ); }
}
```

desert/Oasis.java

```
package com.aov.village.object.desert;

import com.aov.village.object.VillageObject;
import com.aov.village.object.WaterSource;

public class Oasis extends WaterSource {
    public Oasis() { super("Oasis", "Desert"); }

    private Oasis(Oasis other) {
        super("Oasis", "Desert");
        this.children.addAll(other.cloneChildren());
    }

    @Override public VillageObject deepClone() { return new Oasis(this); }
}
```

4.6 builder package – Builder Pattern core

VillageObjectBuilder.java

```
package com.aov.village.builder;

import com.aov.village.object.VillageObject;

/**
 * BUILDER PATTERN
 * ---------------
 * Separates the step-by-step construction of a VillageObject (adding
 * foundation, structure and detail shapes) from its final representation.
```

```
 * Each concrete village object (BrickHouse, MangoTree, Pond, ...) gets
 * its own builder that knows exactly which simple shapes to add and how.
 */
public interface VillageObjectBuilder {
    void reset();
    void buildFoundation();
    void buildStructure();
    void buildDetails();
    VillageObject getResult();
}
```

VillageObjectDirector.java

```
package com.aov.village.builder;

import com.aov.village.object.VillageObject;

/**
 * BUILDER PATTERN - Director
 * --------------------------
 * Knows the correct ORDER of construction steps but nothing about
 * shapes or colors. Any builder can be plugged in.
 */
public class VillageObjectDirector {
    public VillageObject construct(VillageObjectBuilder builder) {
        builder.reset();
        builder.buildFoundation();
        builder.buildStructure();
        builder.buildDetails();
        return builder.getResult();
    }
}
```

4.7 builder.modern package – Modern theme builders

modern/BrickHouseBuilder.java

```
package com.aov.village.builder.modern;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.modern.BrickHouse;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;
import com.aov.village.shape.Square;
import com.aov.village.shape.Triangle;

public class BrickHouseBuilder implements VillageObjectBuilder {
    private BrickHouse house;

    @Override public void reset() { house = new BrickHouse(); }

    @Override public void buildFoundation() {
        house.addShape(new Rectangle(6, 4, "Grey-Concrete"));
    }

    @Override public void buildStructure() {
        house.addShape(new Rectangle(6, 5, "Red-Brick"));
        house.addShape(new Square(1.2, "Brown-Wood-Door"));
```

```
    }

    @Override public void buildDetails() {
        house.addShape(new Triangle(6, 3, "Dark-Red-Roof"));
        house.addShape(new Circle(0.4, "Glass-Window"));
    }

    @Override public VillageObject getResult() { return house; }
}
```

modern/MangoTreeBuilder.java

```
package com.aov.village.builder.modern;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.modern.MangoTree;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;

public class MangoTreeBuilder implements VillageObjectBuilder {
    private MangoTree tree;

    @Override public void reset() { tree = new MangoTree(); }

    @Override public void buildFoundation() {
        tree.addShape(new Rectangle(0.5, 1.5, "Brown-Trunk"));
    }

    @Override public void buildStructure() {
        tree.addShape(new Circle(2.0, "Green-Foliage"));
    }

    @Override public void buildDetails() {
        tree.addShape(new Circle(0.15, "Orange-Mango"));
        tree.addShape(new Circle(0.15, "Orange-Mango"));
        tree.addShape(new Circle(0.15, "Orange-Mango"));
    }

    @Override public VillageObject getResult() { return tree; }
}
```

modern/SwimmingPoolBuilder.java

```
package com.aov.village.builder.modern;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.modern.SwimmingPool;
import com.aov.village.shape.Rectangle;
import com.aov.village.shape.Square;

public class SwimmingPoolBuilder implements VillageObjectBuilder {
    private SwimmingPool pool;

    @Override public void reset() { pool = new SwimmingPool(); }

    @Override public void buildFoundation() {
        pool.addShape(new Rectangle(8, 4, "White-Tile-Deck"));
```

```
    }

    @Override public void buildStructure() {
        pool.addShape(new Rectangle(7, 3, "Aqua-Water"));
    }

    @Override public void buildDetails() {
        pool.addShape(new Square(1, "Blue-Tile-Border"));
    }

    @Override public VillageObject getResult() { return pool; }
}
```

4.8 builder.traditional package – Traditional theme builders

traditional/MudHouseBuilder.java

```
package com.aov.village.builder.traditional;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.traditional.MudHouse;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;
import com.aov.village.shape.Square;
import com.aov.village.shape.Triangle;

public class MudHouseBuilder implements VillageObjectBuilder {
    private MudHouse house;

    @Override public void reset() { house = new MudHouse(); }

    @Override public void buildFoundation() {
        house.addShape(new Rectangle(5, 3, "Brown-Soil-Base"));
    }

    @Override public void buildStructure() {
        house.addShape(new Rectangle(5, 4, "Mud-Wall"));
        house.addShape(new Square(1.0, "Wood-Door"));
    }

    @Override public void buildDetails() {
        house.addShape(new Triangle(5, 2.5, "Straw-Roof"));
        house.addShape(new Circle(0.3, "Small-Window"));
    }

    @Override public VillageObject getResult() { return house; }
}
```

traditional/BananaTreeBuilder.java

```
package com.aov.village.builder.traditional;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.traditional.BananaTree;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;
import com.aov.village.shape.Triangle;
```

```java

public class BananaTreeBuilder implements VillageObjectBuilder {
    private BananaTree tree;

    @Override public void reset() { tree = new BananaTree(); }

    @Override public void buildFoundation() {
        tree.addShape(new Rectangle(0.3, 1.2, "Green-Trunk"));
    }

    @Override public void buildStructure() {
        tree.addShape(new Triangle(1.5, 0.6, "Green-Leaf"));
        tree.addShape(new Triangle(1.5, 0.6, "Green-Leaf"));
        tree.addShape(new Triangle(1.5, 0.6, "Green-Leaf"));
    }

    @Override public void buildDetails() {
        tree.addShape(new Circle(0.25, "Yellow-Banana-Bunch"));
    }

    @Override public VillageObject getResult() { return tree; }
}
```

traditional/PondBuilder.java

```java
package com.aov.village.builder.traditional;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.traditional.Pond;
import com.aov.village.shape.Circle;
import com.aov.village.shape.SemiCircle;

public class PondBuilder implements VillageObjectBuilder {
    private Pond pond;

    @Override public void reset() { pond = new Pond(); }

    @Override public void buildFoundation() {
        pond.addShape(new Circle(3, "Brown-Mud-Edge"));
    }

    @Override public void buildStructure() {
        pond.addShape(new Circle(2.5, "Blue-Water"));
    }

    @Override public void buildDetails() {
        pond.addShape(new SemiCircle(1, "Green-Reeds"));
    }

    @Override public VillageObject getResult() { return pond; }
}
```

4.9 builder.desert package – Desert theme builders

desert/AdobeHouseBuilder.java

```java
package com.aov.village.builder.desert;

```

```java
import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.desert.AdobeHouse;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;
import com.aov.village.shape.Square;

public class AdobeHouseBuilder implements VillageObjectBuilder {
    private AdobeHouse house;

    @Override public void reset() { house = new AdobeHouse(); }

    @Override public void buildFoundation() {
        house.addShape(new Rectangle(5, 3, "Sand-Base"));
    }

    @Override public void buildStructure() {
        house.addShape(new Rectangle(5, 4, "Adobe-Clay-Wall"));
        house.addShape(new Square(1.0, "Wood-Door"));
    }

    @Override public void buildDetails() {
        house.addShape(new Rectangle(5, 0.4, "Flat-Roof-Beam"));
        house.addShape(new Circle(0.3, "Small-Window"));
    }

    @Override public VillageObject getResult() { return house; }
}
```

desert/CactusTreeBuilder.java

```java
package com.aov.village.builder.desert;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.desert.CactusTree;
import com.aov.village.shape.Circle;
import com.aov.village.shape.Rectangle;

public class CactusTreeBuilder implements VillageObjectBuilder {
    private CactusTree tree;

    @Override public void reset() { tree = new CactusTree(); }

    @Override public void buildFoundation() {
        tree.addShape(new Circle(0.5, "Sand"));
    }

    @Override public void buildStructure() {
        tree.addShape(new Rectangle(0.6, 2.5, "Green-Cactus-Body"));
        tree.addShape(new Rectangle(0.3, 0.8, "Green-Cactus-Arm"));
    }

    @Override public void buildDetails() {
        tree.addShape(new Circle(0.1, "Pink-Flower"));
    }

    @Override public VillageObject getResult() { return tree; }
}
```

desert/OasisBuilder.java

```java
package com.aov.village.builder.desert;

import com.aov.village.builder.VillageObjectBuilder;
import com.aov.village.object.VillageObject;
import com.aov.village.object.desert.Oasis;
import com.aov.village.shape.Circle;

public class OasisBuilder implements VillageObjectBuilder {
    private Oasis oasis;

    @Override public void reset() { oasis = new Oasis(); }

    @Override public void buildFoundation() {
        oasis.addShape(new Circle(4, "Sand-Edge"));
    }

    @Override public void buildStructure() {
        oasis.addShape(new Circle(3, "Blue-Water"));
    }

    @Override public void buildDetails() {
        oasis.addShape(new Circle(0.2, "Green-Palm-Leaf"));
        oasis.addShape(new Circle(0.2, "Green-Palm-Leaf"));
    }

    @Override public VillageObject getResult() { return oasis; }
}
```

4.10 factory package – Abstract Factory Pattern

VillageThemeFactory.java

```java
package com.aov.village.factory;

import com.aov.village.object.House;
import com.aov.village.object.Tree;
import com.aov.village.object.WaterSource;

/**
 * ABSTRACT FACTORY PATTERN
 * ------------------------
 * Declares creation methods for a whole FAMILY of related products
 * (House + Tree + WaterSource) that must all belong to the same theme.
 * Adding a new theme = adding one new class that implements this
 * interface; NOTHING here or in any existing factory needs to change.
 */
public interface VillageThemeFactory {
    String getThemeName();
    House createHouse();
    Tree createTree();
    WaterSource createWaterSource();
}
```

ModernVillageFactory.java

```java
package com.aov.village.factory;

```

```java
import com.aov.village.builder.VillageObjectDirector;
import com.aov.village.builder.modern.BrickHouseBuilder;
import com.aov.village.builder.modern.MangoTreeBuilder;
import com.aov.village.builder.modern.SwimmingPoolBuilder;
import com.aov.village.object.House;
import com.aov.village.object.Tree;
import com.aov.village.object.WaterSource;

/** Concrete Factory: Modern Village -> Brick House + Mango Tree + Swimming Pool. */
public class ModernVillageFactory implements VillageThemeFactory {
    private final VillageObjectDirector director = new VillageObjectDirector();

    @Override public String getThemeName() { return "Modern"; }

    @Override public House createHouse() {
        return (House) director.construct(new BrickHouseBuilder());
    }

    @Override public Tree createTree() {
        return (Tree) director.construct(new MangoTreeBuilder());
    }

    @Override public WaterSource createWaterSource() {
        return (WaterSource) director.construct(new SwimmingPoolBuilder());
    }
}
```

TraditionalVillageFactory.java

```java
package com.aov.village.factory;

import com.aov.village.builder.VillageObjectDirector;
import com.aov.village.builder.traditional.BananaTreeBuilder;
import com.aov.village.builder.traditional.MudHouseBuilder;
import com.aov.village.builder.traditional.PondBuilder;
import com.aov.village.object.House;
import com.aov.village.object.Tree;
import com.aov.village.object.WaterSource;

/** Concrete Factory: Traditional Village -> Mud House + Banana Tree + Pond. */
public class TraditionalVillageFactory implements VillageThemeFactory {
    private final VillageObjectDirector director = new VillageObjectDirector();

    @Override public String getThemeName() { return "Traditional"; }

    @Override public House createHouse() {
        return (House) director.construct(new MudHouseBuilder());
    }

    @Override public Tree createTree() {
        return (Tree) director.construct(new BananaTreeBuilder());
    }

    @Override public WaterSource createWaterSource() {
        return (WaterSource) director.construct(new PondBuilder());
```

```
    }
}
```

DesertVillageFactory.java

```
package com.aov.village.factory;

import com.aov.village.builder.VillageObjectDirector;
import com.aov.village.builder.desert.AdobeHouseBuilder;
import com.aov.village.builder.desert.CactusTreeBuilder;
import com.aov.village.builder.desert.OasisBuilder;
import com.aov.village.object.House;
import com.aov.village.object.Tree;
import com.aov.village.object.WaterSource;

/**
 * Concrete Factory: Desert Village -> Adobe House + Cactus Tree + Oasis.
 * NOTE: this whole file is the ONLY new code required to add a brand-new
 * theme to the game; VillageThemeFactory, the Registry, the Facade and
 * every existing factory/builder/product class are untouched (Open/Closed
 * Principle in action).
 */
public class DesertVillageFactory implements VillageThemeFactory {
    private final VillageObjectDirector director = new
        VillageObjectDirector();

    @Override public String getThemeName() { return "Desert"; }

    @Override public House createHouse() {
        return (House) director.construct(new AdobeHouseBuilder());
    }

    @Override public Tree createTree() {
        return (Tree) director.construct(new CactusTreeBuilder());
    }

    @Override public WaterSource createWaterSource() {
        return (WaterSource) director.construct(new OasisBuilder());
    }
}
```

4.11 registry package – Singleton Pattern

VillageFactoryRegistry.java

```
package com.aov.village.registry;

import com.aov.village.factory.VillageThemeFactory;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

/**
 * SINGLETON PATTERN + Registry
 * ----------------------------
 * One global, lazily-created access point that maps a theme name
 * ("Modern", "Traditional", "Desert", ...) to its VillageThemeFactory.
```

```
 * New themes are plugged in at the composition root (see Main.java)
 * via registerFactory(); this class itself never needs to change.
 */
public final class VillageFactoryRegistry {
    private static volatile VillageFactoryRegistry instance;
    private final Map<String, VillageThemeFactory> factories = new HashMap
        <>();

    private VillageFactoryRegistry() { }

    public static VillageFactoryRegistry getInstance() {
        if (instance == null) {
            synchronized (VillageFactoryRegistry.class) {
                if (instance == null) {
                    instance = new VillageFactoryRegistry();
                }
            }
        }
        return instance;
    }

    public void registerFactory(VillageThemeFactory factory) {
        factories.put(factory.getThemeName().toLowerCase(), factory);
    }

    public VillageThemeFactory getFactory(String themeName) {
        VillageThemeFactory factory = factories.get(themeName.toLowerCase()
            );
        if (factory == null) {
            throw new IllegalArgumentException("No factory registered for
                theme: " + themeName);
        }
        return factory;
    }

    public Set<String> getAvailableThemes() {
        return Collections.unmodifiableSet(factories.keySet());
    }
}
```

4.12 render package – Strategy Pattern

RenderStrategy.java

```
package com.aov.village.render;

import com.aov.village.object.VillageObject;

/**
 * STRATEGY PATTERN
 * ----------------
 * Decouples HOW a village object gets displayed/rendered from the
 * object itself. The game could later add a GraphicalRenderStrategy
 * or JsonExportRenderStrategy without touching VillageObject at all.
 */
public interface RenderStrategy {
    void render(VillageObject object);
}
```

ConsoleRenderStrategy.java

```java
package com.aov.village.render;

import com.aov.village.object.VillageObject;

/** Concrete Strategy: prints the object tree straight to the console. */
public class ConsoleRenderStrategy implements RenderStrategy {
    @Override
    public void render(VillageObject object) {
        System.out.println("[" + object.getType() + "] " + object.getName()
            + " (" + object.getTheme() + " theme)");
        object.draw(1);
        System.out.printf("  Total footprint area: %.2f sq. units%n",
           object.getArea());
    }
}
```

4.13 service package – Facade Pattern

Village.java

```java
package com.aov.village.service;

import com.aov.village.object.VillageObject;
import com.aov.village.render.RenderStrategy;

import java.util.ArrayList;
import java.util.List;

/** A player's village: a name, a theme, and the objects placed in it. */
public class Village {
    private final String name;
    private final String theme;
    private final List<VillageObject> objects = new ArrayList<>();

    public Village(String name, String theme) {
        this.name = name;
        this.theme = theme;
    }

    public void addObject(VillageObject object) { objects.add(object); }

    public String getName() { return name; }
    public String getTheme() { return theme; }
    public List<VillageObject> getObjects() { return objects; }

    public void render(RenderStrategy strategy) {
        System.out.println("===== Village: " + name + " (Theme: " + theme +
            ") =====");
        for (VillageObject obj : objects) {
            strategy.render(obj);
            System.out.println();
        }
    }
}
```

VillageCreationService.java

```java
package com.aov.village.service;

import com.aov.village.factory.VillageThemeFactory;
import com.aov.village.registry.VillageFactoryRegistry;

/**
 * FACADE PATTERN
 * ---------------
 * Hides the Registry + Abstract Factory + Builder + Director machinery
 * behind one simple call so client code (UI, game engine, tests, ...)
 * doesn't need to know any of the internal wiring.
 */
public class VillageCreationService {
    private final VillageFactoryRegistry registry = VillageFactoryRegistry.
        getInstance();

    public Village createVillage(String villageName, String themeName) {
        VillageThemeFactory factory = registry.getFactory(themeName);
        Village village = new Village(villageName, factory.getThemeName());
        village.addObject(factory.createHouse());
        village.addObject(factory.createTree());
        village.addObject(factory.createWaterSource());
        return village;
    }
}
```

4.14 app package – Demo / composition root

Main.java

```java
package com.aov.village.app;

import com.aov.village.factory.DesertVillageFactory;
import com.aov.village.factory.ModernVillageFactory;
import com.aov.village.factory.TraditionalVillageFactory;
import com.aov.village.object.VillageObject;
import com.aov.village.registry.VillageFactoryRegistry;
import com.aov.village.render.ConsoleRenderStrategy;
import com.aov.village.render.RenderStrategy;
import com.aov.village.service.Village;
import com.aov.village.service.VillageCreationService;

/**
 * Demo / composition root for Age of Villagers (AoV) village creation
    module.
 */
public class Main {
    public static void main(String[] args) {
        // ---- Composition root: register the themes available today (
            Singleton Registry) ----
        VillageFactoryRegistry registry = VillageFactoryRegistry.
            getInstance();
        registry.registerFactory(new ModernVillageFactory());
        registry.registerFactory(new TraditionalVillageFactory());

        // Extensibility proof: a brand-new "Desert" theme is added here
            with
        // ONE line, and DesertVillageFactory.java was the only new file
            needed.
```

```
25          // No existing class (Registry, Facade, other factories/builders/
                products)
26          // was modified. This satisfies the assignment's Open/Closed
                requirement.
27          registry.registerFactory(new DesertVillageFactory());
28
29          VillageCreationService villageService = new VillageCreationService
                ();
30          RenderStrategy renderer = new ConsoleRenderStrategy();
31
32          Village modernVillage = villageService.createVillage("Green Meadows
                ", "Modern");
33          Village traditionalVillage = villageService.createVillage("Old Town
                ", "Traditional");
34          Village desertVillage = villageService.createVillage("Sandy Dunes",
                 "Desert");
35
36          modernVillage.render(renderer);
37          traditionalVillage.render(renderer);
38          desertVillage.render(renderer);
39
40          // ---- PROTOTYPE PATTERN demo: clone an existing tree instead of
                rebuilding it ----
41          VillageObject originalTree = modernVillage.getObjects().get(1); //
                Mango Tree
42          VillageObject clonedTree = originalTree.deepClone();
43          modernVillage.addObject(clonedTree);
44          System.out.println("Cloned a \"" + clonedTree.getName() + "\" via
                the Prototype pattern. \""
45                  + modernVillage.getName() + "\" now has " + modernVillage.
                      getObjects().size() + " objects.");
46
47          System.out.println();
48          System.out.println("Registered themes in the game: " + registry.
                getAvailableThemes());
49      }
50  }
```

5 Project / Folder Structure

```
AoV/
|-- README.md
`-- src/
    `-- com/aov/village/
        |-- shape/                      Simple shapes + Composite root
        |   |-- Shape.java
        |   |-- ShapeUtils.java
        |   |-- Rectangle.java
        |   |-- Square.java
        |   |-- Triangle.java
        |   |-- Circle.java
        |   |-- SemiCircle.java
        |   `-- CompositeShape.java
        |
        |-- object/                     Village-object hierarchy (products)
        |   |-- VillageObject.java
        |   |-- House.java
        |   |-- Tree.java
        |   |-- WaterSource.java
        |   |-- modern/
        |   |   |-- BrickHouse.java
```

```
        |   |   |-- MangoTree.java
        |   |   `-- SwimmingPool.java
        |   |-- traditional/
        |   |   |-- MudHouse.java
        |   |   |-- BananaTree.java
        |   |   `-- Pond.java
        |   `-- desert/                (proof of extensibility)
        |       |-- AdobeHouse.java
        |       |-- CactusTree.java
        |       `-- Oasis.java
        |
        |-- builder/                   Step-by-step shape assembly
        |   |-- VillageObjectBuilder.java
        |   |-- VillageObjectDirector.java
        |   |-- modern/    (BrickHouseBuilder, MangoTreeBuilder, SwimmingPoolBuilder)
        |   |-- traditional/ (MudHouseBuilder, BananaTreeBuilder, PondBuilder)
        |   `-- desert/    (AdobeHouseBuilder, CactusTreeBuilder, OasisBuilder)
        |
        |-- factory/                   Abstract Factory (theme families)
        |   |-- VillageThemeFactory.java
        |   |-- ModernVillageFactory.java
        |   |-- TraditionalVillageFactory.java
        |   `-- DesertVillageFactory.java
        |
        |-- registry/                  Singleton lookup of factories
        |   `-- VillageFactoryRegistry.java
        |
        |-- render/                    Strategy for displaying objects
        |   |-- RenderStrategy.java
        |   `-- ConsoleRenderStrategy.java
        |
        |-- service/                   Facade for client code
        |   |-- Village.java
        |   `-- VillageCreationService.java
        |
        `-- app/
            `-- Main.java               Demo / composition root
```

Compile & Run

```
find src -name "*.java" > sources.txt
javac -d out @sources.txt
java -cp out com.aov.village.app.Main
```

This project was compiled and executed successfully with OpenJDK 21. Running `Main` prints all three villages (Modern, Traditional, Desert) with every shape and computed area, then demonstrates the Prototype pattern by cloning a Mango Tree, and finally prints the set of registered themes.

6 Repository Link

GitHub Link:

`https://github.com/Hasib-39/Age-of-Villagers`

7 Conclusion

The Age of Villagers village-creation module was implemented using the Abstract Factory, Builder, Composite, Singleton, Facade, Strategy, and Prototype patterns working together. The Abstract Factory guarantees that every theme produces a consistent family of objects; the Builder assembles each object from simple shapes step-by-step; the Composite lets shapes and whole objects be treated

uniformly; the Singleton registry provides one global lookup of themes; the Facade gives client code a single simple entry point; the Strategy decouples rendering from the objects themselves; and the Prototype allows cheap duplication of existing objects. Together, these patterns satisfy the requirement that entirely new village themes – proven concretely with a working Desert theme – can be added without modifying any existing class.